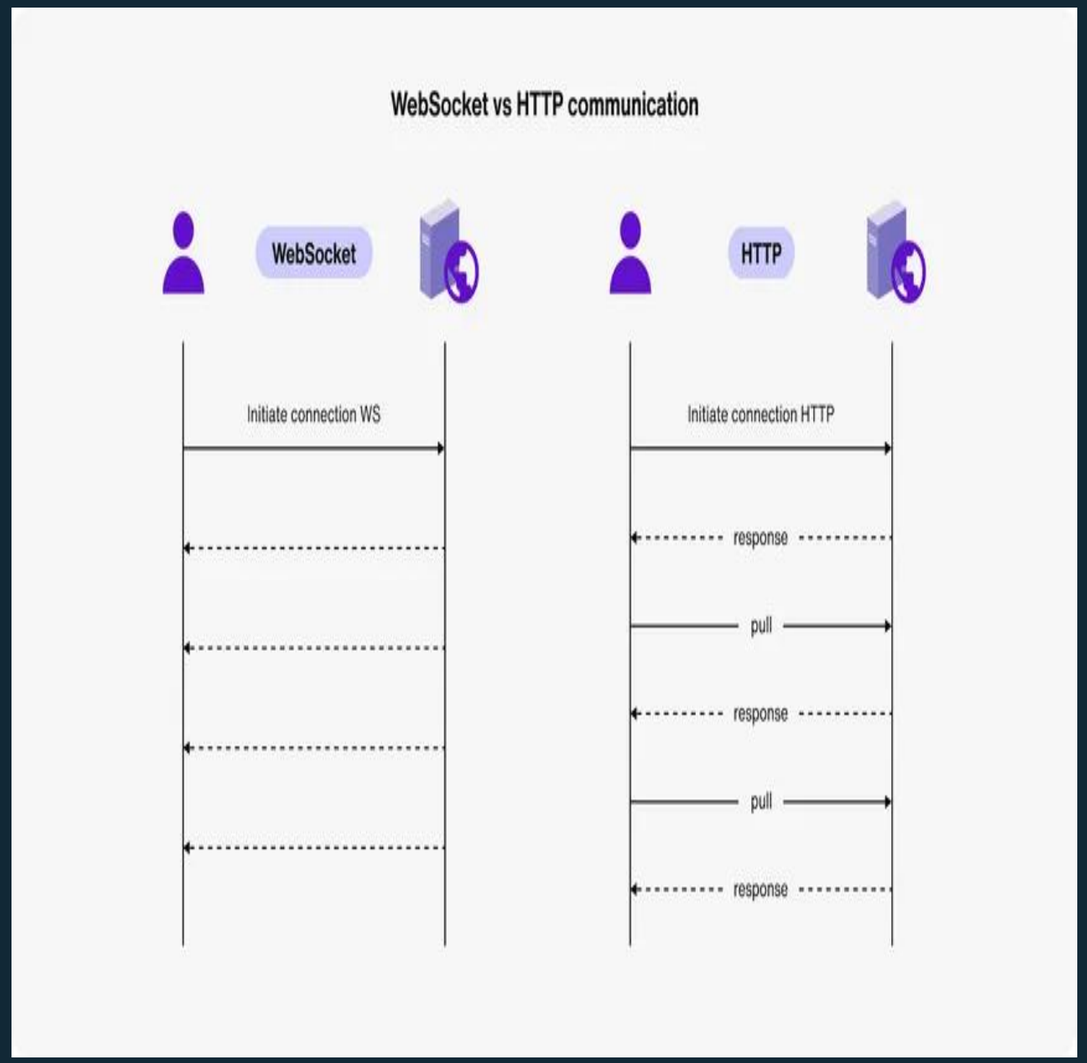
Real-Time Chat Application using FastAPI & WebSockets

Infosys Springboard Internship — concise technical overview, architecture, flows, and demo-ready assets. Presented for internship evaluators and technical peers.



Problem: Limitations of Request/Response

Traditional HTTP: client polls server periodically. Results: higher latency, wasted bandwidth, and poor UX for live interaction. WebSockets provide persistent bi-directional channels to eliminate polling.

Objective

Real-time Communication

Low-latency, event-driven message delivery between clients and server.

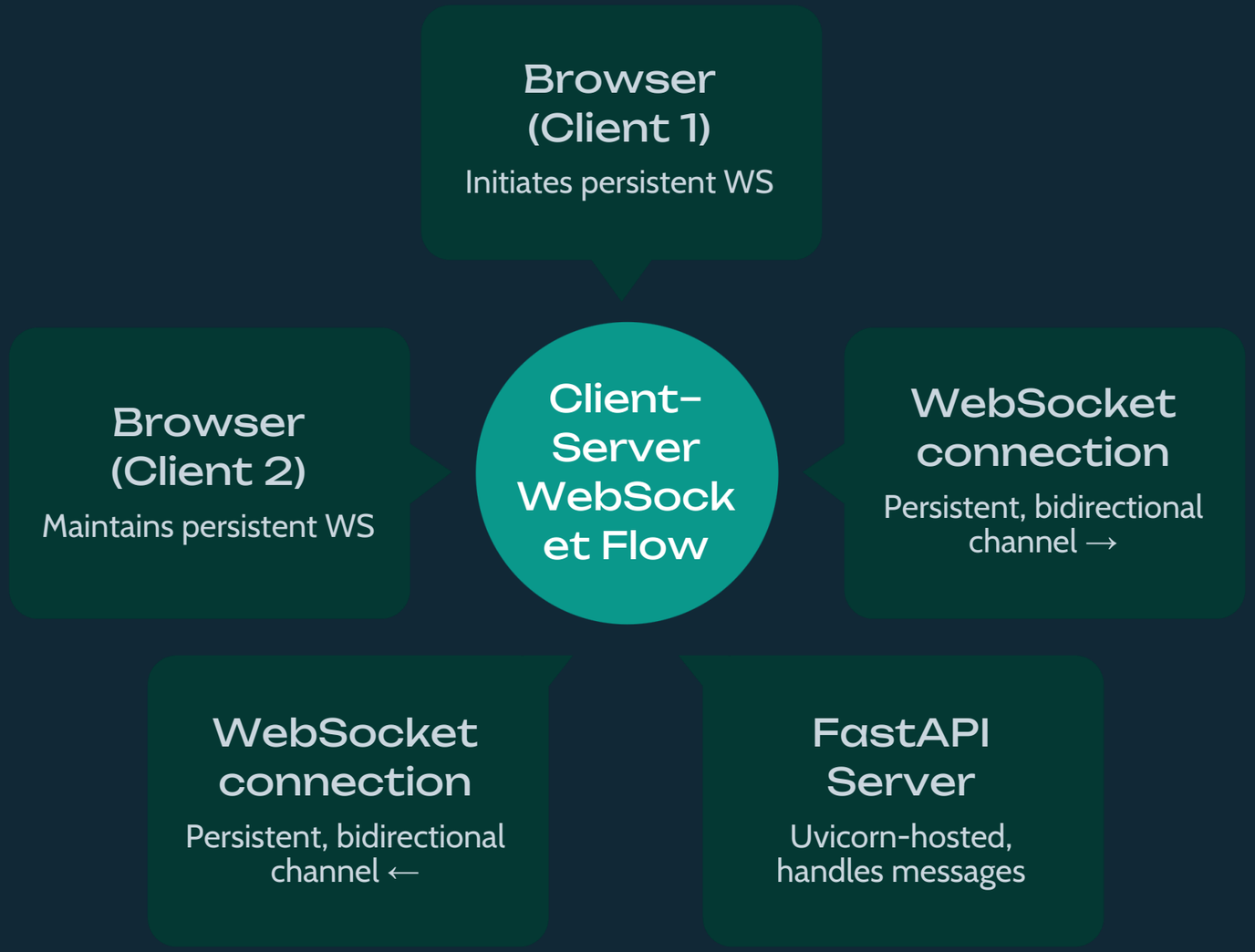
Room-based Chat

Isolated channels (rooms) for scoped broadcasts and membership management.

Structured JSON Messages

Consistent message schema for routing, typing, and client handling.

Client-Server Architecture

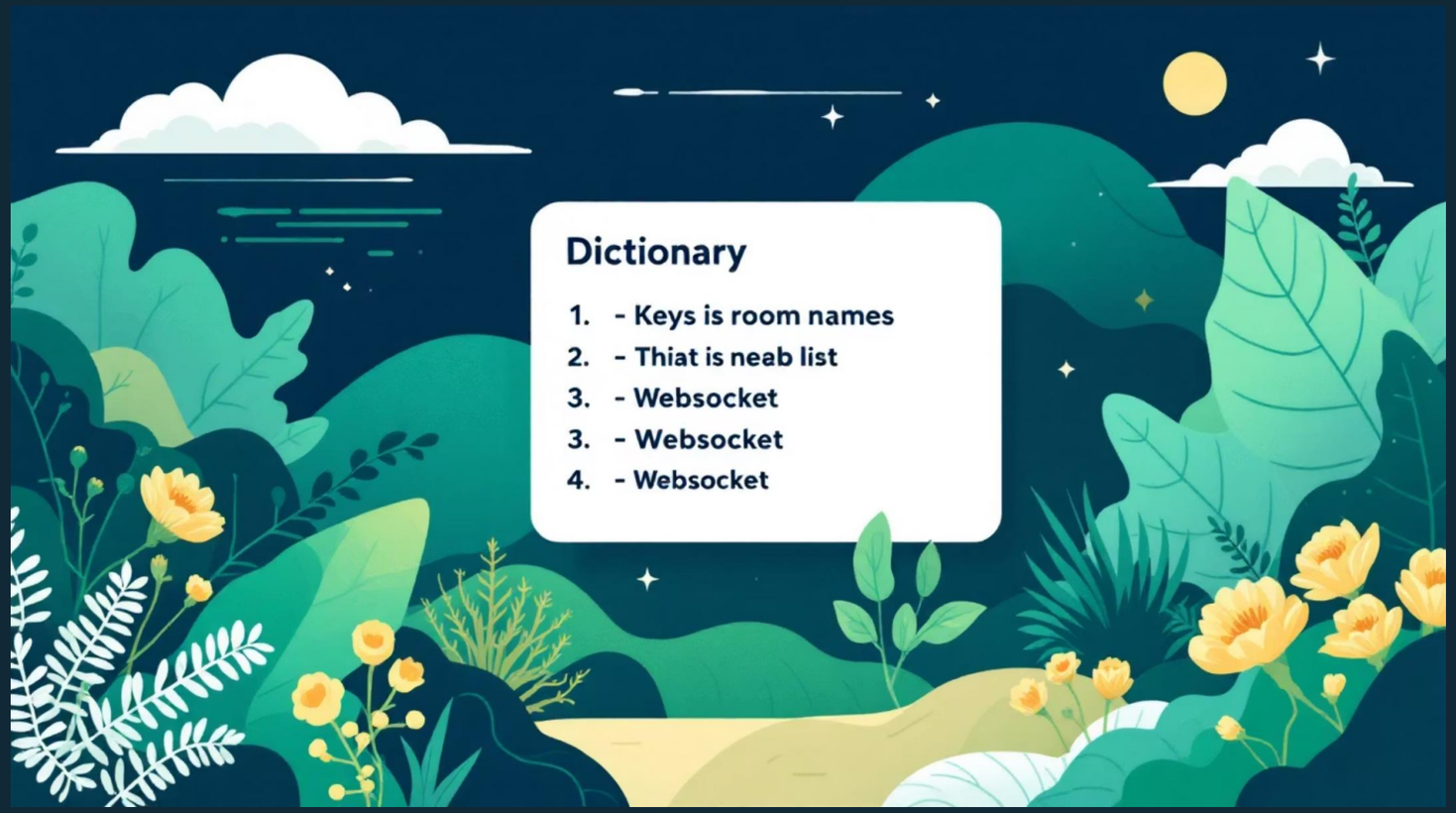


This architecture uses Uvicorn to host FastAPI. Each browser maintains a persistent WebSocket to the server for immediate message exchange.

WebSocket Communication Flow

Sequence captures the typical session lifecycle. Authentication is optional but recommended for production.

Room Management



Implementation notes:

- Rooms stored as dict: room_name → set/list of active websocket connections.
- Join/leave mutate the collection; server iterates collection to broadcast.
- Graceful disconnect removes socket to avoid stale references.

Message Format (JSON)

1

type

Action category (e.g., chat, join, leave, system).

2

sender

Client identifier for display and logging.

3

room

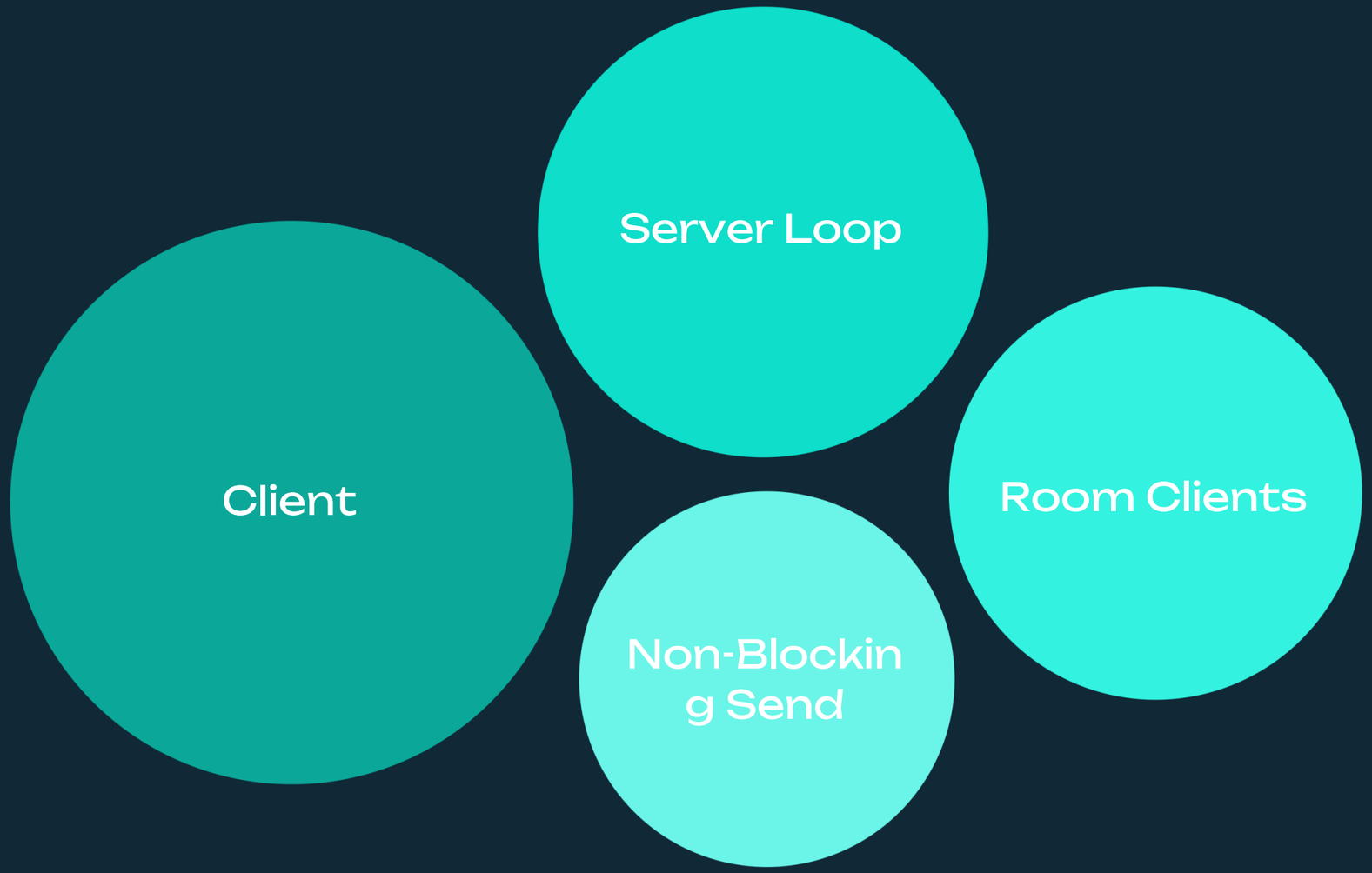
Destination channel for routing on server side.

4

message / timestamp

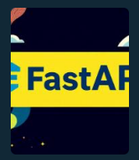
Payload and ISO-8601 timestamp for ordering and audit.

Broadcasting Workflow



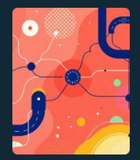
Key considerations: async non-blocking sends, exception handling per socket, and backpressure management to prevent server stalls.

Implementation Snapshot & Tech Stack



FastAPI

Asynchronous endpoints and WebSocket routing; high-performance Python framework.



WebSocket

Persistent, bi-directional protocol for live messaging.

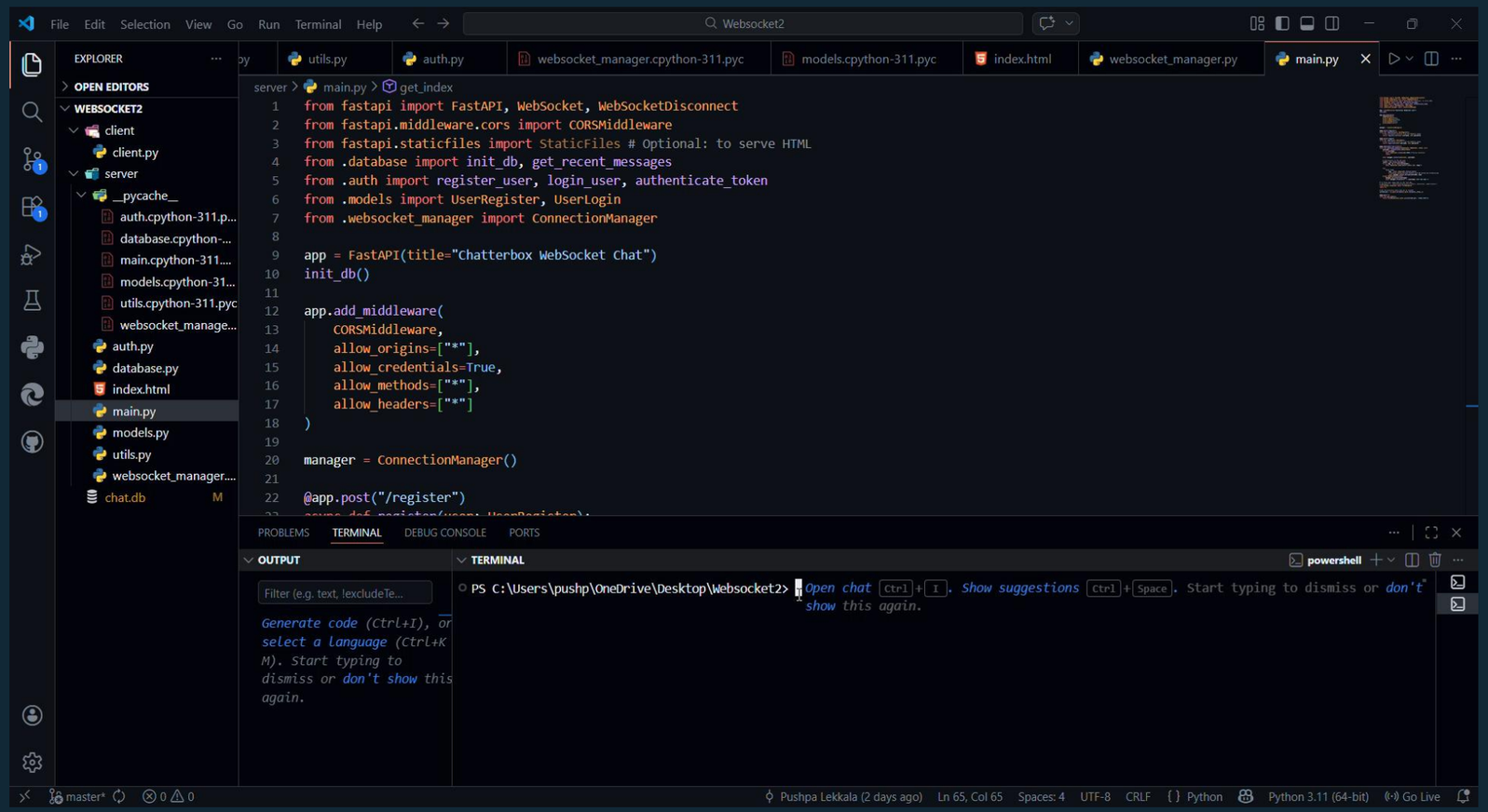
Python & Uvicorn

Async runtime (uvicorn) for event loop and concurrency.

JSON

Compact structured payloads for interoperability and validation.

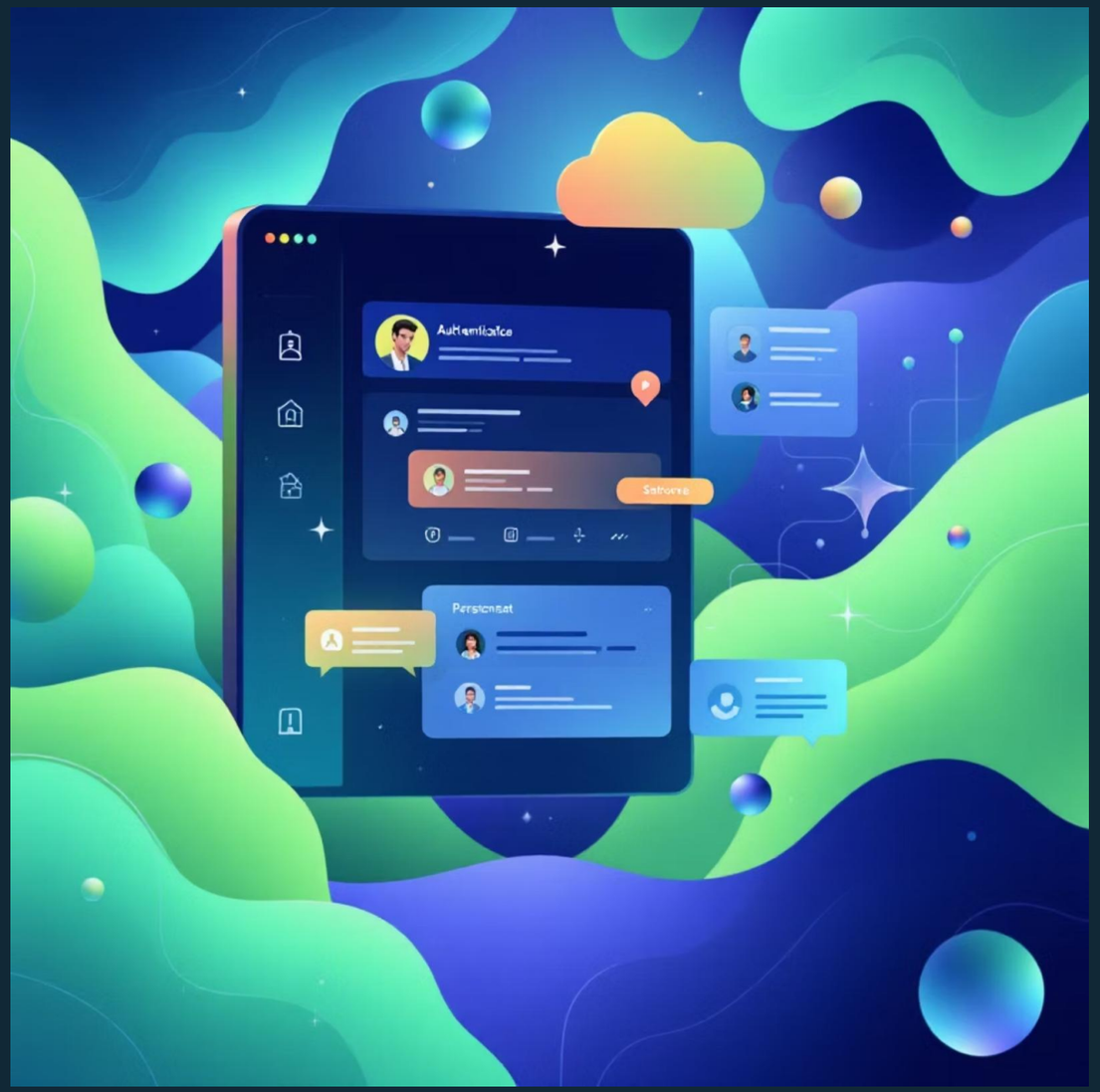
Demo & Next Steps



Demo Instructions

- Start server: uvicorn app:app --reload
- Open two browser windows; connect to WebSocket endpoint
- Send messages and observe immediate broadcast to room members

Future Scope



Future Enhancements :

- **Auth:** Secure login and token-based user authentication system.
- **Persistent Storage:** Store chats reliably using database integration.
- **File Sharing:** Enable users to upload and share files.
- **UI Polish:** Improve interface design and user experience.
- **Containerized Deployment:** Deploy application using Docker containers.

Uses of the Project :

- **Real-time Group Communication Systems**: Enable instant messaging among multiple connected users.
- **Collaborative Document Editing Platforms**: Allow multiple users to edit documents simultaneously.
- **Multiplayer Online Gaming Backends**: Support real-time player interaction and synchronization.
- **Live Data Streaming Dashboards**: Deliver real-time updates without page refresh.
- **Instant Notification Broadcasting Systems**: Push live alerts to all connected clients.

WEB CHAT

Limitations of the Project :

- **Requires Constant Internet Connection:** System depends on continuous network connectivity.
- **Scalability Issues:** Performance may degrade with increasing concurrent users.
- **Single Server Failure:** Centralized server crash affects all connected clients.
- **Security Vulnerabilities:** Risks increase without proper authentication and encryption.
- **No Offline Support:** Messages unavailable without persistent storage mechanisms.

Conclusion

- **Project Overview:** Developed a real-time messaging system using WebSockets and FastAPI.
- **Instant Communication:** Enables seamless interaction between multiple connected users.
- **Architecture Implementation:** Demonstrates practical real-time server design concepts.
- **Future Enhancements:** Can be improved with authentication, storage, and deployment features.
- **Scalable Foundation:** Serves as a strong base for advanced real-time applications.